

The ICF Playbook

CodeSignal Industry Coding Framework — strategy for one attempt

August 2026

Contents

The ICF Playbook	1
1. What the format actually is	1
2. The two patterns that decide your score	2
3. The universal Level 1 skeleton	3
4. Minute-by-minute plan	4
5. Rules from the official brief	5
6. Domain archetypes	5
7. The five mistakes that cost the most	5
8. Integrity	6
9. Two-hour drill using this playbook	6

The ICF Playbook

How to attack a CodeSignal **Industry Coding Framework** assessment. Built from CodeSignal's own technical brief and support documentation, not from candidate rumor. One attempt, 90 minutes — this is the strategy document.

Sources: [ICA rules](#) and the [Industry Coding Skills Evaluation Framework technical brief](#).

1. What the format actually is

One project-based task, four progressive levels, 90 minutes, score 200-600. You are **not expected to finish**. Partial credit is granted per test passed.

CodeSignal's official per-level spec — memorize this table, it tells you what kind of thinking each level wants:

Level	Name	What it asks	Methods	Cumulative LOC	Suggested time
1	Initial Design & Basic Functions	Basic ops, corner cases, error handling	3-4	15-20	10-15 min
2	Data Structures & Data Processing	Filtering, aggregation, top-N, export	1-2 more	30-45	20-30 min

Level	Name	What it asks	Methods	Cumulative LOC	Suggested time
3	Refactoring & Encapsulation	Extend existing methods with new capability	3-5 more	90-130	30-60 min
4	Extending Design & Functionality	Final capability, backward compatible	1-2 more	110-160	30-60 min

Two things jump out. First, **Level 3 is the big one** — the most new methods and the largest code jump (45 → 130 lines). Second, the whole solution is only ~150 lines. This is not a volume test.

Explicitly excluded by CodeSignal

The brief states these are **out of scope at every level**:

- Complex or niche algorithms — binary search, two pointers, dynamic programming
- Advanced data structures you'd implement yourself (trees, tries, heaps by hand)
- Optimization work
- Third-party libraries (standard library only)
- Concurrency, parallelism, distributed computing
- Parsing data files

So: no LeetCode. What *is* in scope is dicts, lists, sorting, collections, and clean class design. If you find yourself writing a clever algorithm, you have misread the problem.

2. The two patterns that decide your score

CodeSignal's own published example (a file hosting service) shows the canonical shape of Levels 3 and 4. These recur across ICF problems because they're what "refactoring & encapsulation" and "backward-compatible extension" naturally look like.

Pattern A — Level 3 is almost always "now with time"

In the official example, Level 3 takes every Level 1-2 method and adds a timestamped variant with optional TTL:

```
Level 1:  FILE_UPLOAD(file_name, size)
Level 3:  FILE_UPLOAD_AT(timestamp, file_name, size)
          FILE_UPLOAD_AT(timestamp, file_name, size, ttl)
          FILE_GET_AT(timestamp, file_name)
          FILE_COPY_AT(timestamp, source, dest)
          FILE_SEARCH_AT(timestamp, prefix)    # only "alive" files
```

The instruction is explicit: these *"inherit all functionality"* of the originals. That means the correct implementation is **not** to write five new methods — it's to make your originals delegate to time-aware internals.

Pattern B — Level 4 is almost always “now with history”

The official example’s Level 4 is a single method:

```
ROLLBACK(timestamp)  # restore state to that timestamp; recalculate all TTLs
```

Rollback, undo, restore, snapshot, audit — all the same requirement: **you must be able to reconstruct past state**. If Levels 1-3 mutated data in place and threw away the old values, you cannot do this without a rewrite you don’t have time for.

The consequence for Level 1

You now know, before you read your actual problem, that time and history are probably coming. That’s the entire edge this playbook gives you.

3. The universal Level 1 skeleton

Write Level 1 in this shape regardless of the domain. It costs about three extra minutes and makes Levels 3-4 additive instead of structural.

```
from dataclasses import dataclass, field
from typing import Optional

@dataclass
class Record:
    """Whatever the domain's 'thing' is. NOT a bare value in a dict."""
    key: str
    value: int
    created_at: int = 0
    ttl: Optional[int] = None          # None = lives forever

    def alive_at(self, t: int) -> bool:
        return self.ttl is None or t < self.created_at + self.ttl

class System:
    def __init__(self):
        self.store: dict[str, Record] = {}
        self.history: list[tuple] = []    # (timestamp, op, args...)

    # ---- public API: the signatures the tests call -----
    def set(self, key, value):
        return self._set_at(0, key, value)    # delegate

    def get(self, key):
        return self._get_at(0, key)

    # ---- internals: everything takes a timestamp -----
    def _set_at(self, t, key, value, ttl=None):
        self.history.append((t, "set", key, value, ttl))
        self.store[key] = Record(key, value, created_at=t, ttl=ttl)
        return True

    def _get_at(self, t, key):
        rec = self.store.get(key)
        if rec is None or not rec.alive_at(t):
```

```
    return None
    return rec.value
```

Four decisions are doing the work:

1. **A Record class, not a raw value.** When Level 3 says “files now have a TTL,” you add one field. If you’d stored {key: value}, you’d be rewriting every method.
2. **Every internal takes a timestamp,** even at Level 1 where you pass 0. Level 3’s `_AT` methods then become thin public wrappers.
3. **Public methods delegate to internals.** Level 3 asks you to add methods *without breaking the old ones* — delegation gives you that for free.
4. **An append-only history list.** Costs one line per mutation at Level 1. Buys you Level 4’s rollback.

When Level 3 arrives, you write:

```
def set_at(self, timestamp, key, value, ttl=None):
    return self._set_at(timestamp, key, value, ttl)
```

That’s the entire level for that method. When Level 4 arrives:

```
def rollback(self, timestamp):
    ops = [h for h in self.history if h[0] <= timestamp]
    self.store.clear()
    self.history.clear()
    for t, op, *args in ops:
        getattr(self, f"_{op}_at")(t, *args)    # replay
```

Replaying the log is usually simpler and less bug-prone than trying to undo operations in reverse.

4. Minute-by-minute plan

Clock	Do
0:00–0:08	Read everything. The whole problem, all four levels if visible, and every unit test. Write nothing.
0:08–0:12	Sketch the Record fields and the class skeleton above. Still no real logic.
0:12–0:25	Level 1. Submit as soon as one method works — bank partial credit.
0:25–0:45	Level 2. Filtering/aggregation/top-N. Submit repeatedly.
0:45–1:15	Level 3. Delegate, don’t duplicate. Submit after each method.
1:15–1:30	Level 4 if reached; otherwise re-submit and make sure earlier levels are all green.

Submit often. CodeSignal grants partial credit per test passed, so an unsubmitted correct method scores nothing. There is no penalty for submitting.

If you’re stuck at minute 60 on Level 3, stop and verify Levels 1–2 are fully green. Complete lower levels beat partial higher ones.

5. Rules from the official brief

- **“Do not change the existing method signatures.”** Stated in the task instructions. Extend by adding methods and internal parameters, never by altering a signature the tests already call.
- **“Read the question all the way through before you start coding, but implement the operations and complete the levels one by one.”** Read wide, build narrow.
- **“Please check tests for requirements and argument types.”** The tests are part of the spec, not just a grader. When prose and tests disagree, the tests win.
- Corner cases and error handling are named in Level 1’s criteria. Missing keys, duplicate keys, empty inputs, and “throws if not found” behaviors are scored. Read whether the spec wants an exception, None, or False.

6. Domain archetypes

The task is domain-agnostic by design, so the domain is a costume. Recognize the underlying shape and the levels become predictable.

Domain	L1	L2	L3	L4
File hosting (official example)	upload / get / copy	search by prefix, top-10 by size	_AT variants + TTL	rollback to timestamp
In-memory KV store	set / get / delete	scan by prefix, filtered scan	_AT variants + TTL expiry	backup / restore, or rollback
Banking	create account / deposit / transfer	top-N by transaction volume	scheduled or timestamped payments, cashback	merge accounts, keeping history
Cloud storage / users	add user, add file	list by size/type	per-user quotas over time	merge users, transfer ownership
Inventory / warehouse	add / remove / query item	aggregate by category	reservations with expiry	audit trail, restore
Job scheduler	submit / cancel / status	list by state, counts	timed execution windows	dependency or retry logic
Inference engine (reported for Fellows)	submit request, prefill/decode	metrics, aggregation	batching under memory limits	preemption / load balancing

Note the column pattern: **L2 is always “report on the data,” L3 is always “the same operations but time-aware,” L4 is always “reconstruct or reorganize state.”** Even the inference-engine variant fits — its L3/L4 are memory limits and preemption, which are time-and-state problems in a different suit.

7. The five mistakes that cost the most

1. **Bare dicts and free functions at Level 1.** It works, it’s fast, and it guarantees a rewrite at Level 3. Use a class and a record type.

2. **Duplicating logic in Level 3 instead of delegating.** If your `get()` and `get_at()` both contain the lookup rules, you now maintain two copies and they will drift.
 3. **Mutating in place with no history.** Kills Level 4 outright.
 4. **Not submitting until the level is “done.”** Partial credit is per test.
 5. **Rewriting from scratch when Level 3 hurts.** With 30 minutes left, a targeted extraction of one method beats a rewrite every time.
-

8. Integrity

Reported for Anthropic specifically: submissions are analyzed for test-gaming — hardcoded outputs, branching on test-specific values, structural similarity to circulated solutions. Write code that implements the spec, with meaningful names and honest structure. The skeleton in section 3 is an architecture pattern, not a canned answer; you still have to solve the actual problem with it.

No AI assistance is permitted during the assessment. Practice the same way.

9. Two-hour drill using this playbook

1. **(40 min)** Take the official file-hosting example in section 2 and implement all four levels yourself, from the spec lines alone. It’s the one problem you know is representative, and CodeSignal published it.
2. **(40 min)** Do `mock_oa_kv/` — the in-memory KV store mock, built in ICF house style with the L3 timestamp and L4 rollback patterns.
3. **(40 min)** Do `mock_oa/` — the inference engine mock, matching the domain your cohort reported.

If you only have time for one: the KV store, because its L3/L4 patterns are the ones the official brief says to expect.